LAMHA

Urdu for “moment” — pronounced “lamha”

A time-aware, self-correcting RAG system for real-world regulatory compliance — built to answer in the language people actually type in.

Flagship build: a freelance-tax & regulatory-compliance assistant for Pakistan’s freelancer and small-business community — answering in Urdu, Roman Urdu, and English, grounded in the actual (and constantly amended) law.

Prepared for: Muhammad Abdullah Haroon (“Patti”)
FAST-NUCES Lahore — CS | Target track: LLM / Agentic AI Engineering
Project blueprint | September 2026

01 The problem, stated plainly

Pakistan has one of the largest freelance workforces in the world. Most of that income runs into the same wall every year: FBR tax law, SRO notifications, and State Bank rules on IT export remittances are written in dense statutory English, amended multiple times a year, and almost never explained in the language freelancers actually think and search in. What fills the gap today is Facebook groups, WhatsApp forwards, and YouTube comment sections — confident, often wrong, and almost never dated. Getting it wrong costs real money: missed exemptions, wrong tax regime elections, penalties for late or incorrect filing.

That is the shape of the problem this project targets: a domain where the documents are public, dense, and legally load-bearing; where the true answer changes over time as clauses get amended or superseded; and where real queries arrive code-mixed — "*freelancer ki income pe tax kitna lagta hai agar annual 15 lac se kam ho*" — not the clean, single-language questions most RAG demos are built and tested on.

That combination — high stakes, moving ground truth, and messy multilingual input — is exactly the kind of problem a production RAG system has to survive, and exactly what a portfolio project that only answers clean English questions about a static PDF never has to face.

02 Why this isn't "just another RAG"

Most RAG portfolio projects stop at the part that's easiest to demo: get text into a vector database, embed a question, pull back the top-k chunks, hand them to an LLM. That includes the OCR-to-vector-DB pattern (scan documents, embed, retrieve) — OCR solves an input problem, not a reasoning problem. It also includes the retrieve-then-generate agent built during the Optimus Automate internship. Both are real and useful, but a hiring manager who has seen forty LangChain tutorials will recognize the pattern in one glance and move on.

LAMHA is built around three things that a naive pipeline doesn't handle, on purpose:

1. Time is a first-class citizen, not an afterthought.

Every chunk carries `effective_from` / `effective_to` metadata. Retrieval filters by "currently in force" by default, and the generation step is required to flag when a clause it's citing has since been amended or superseded — instead of confidently quoting a repealed rule. On the performance side, a semantic cache means the huge volume of near-duplicate FAQ-style questions gets served in well under a second instead of re-running the full pipeline every time.

2. It knows when it doesn't know, and does something about it.

A Corrective-RAG grading step scores every retrieved chunk for relevance before generation is allowed to happen. Low-confidence retrieval triggers query rewriting and, if needed, a scoped web-search fallback for very recent circulars that haven't been ingested yet — clearly labelled as unverified against the primary corpus. A naive RAG pipeline generates regardless of whether it actually found anything useful; this one doesn't get that option.

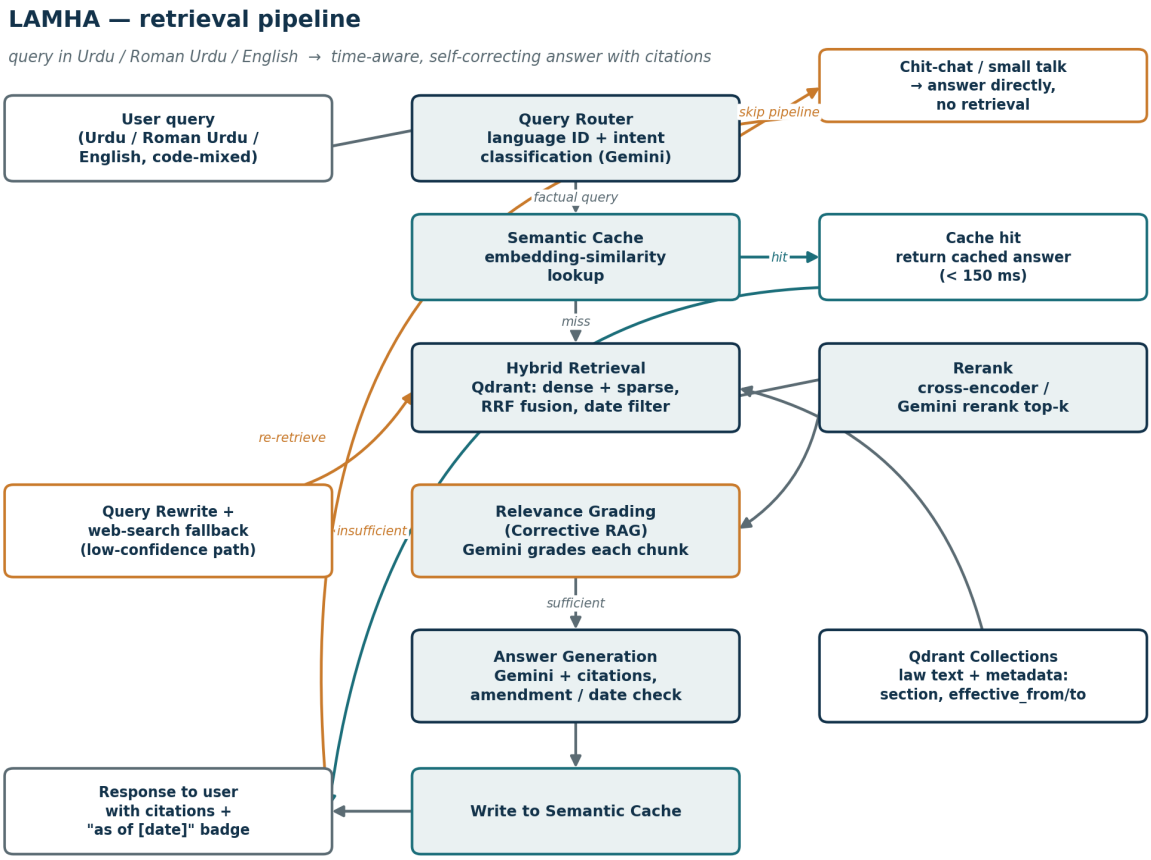
3. Multilingual robustness is designed in, not bolted on.

This is the part that comes straight out of existing research rather than a tutorial: the co-authored paper on domain robustness of multilingual NLP models across Urdu and Roman Urdu scripts studies exactly this failure mode — script-shift, code-mixing intensity, cross-lingual transfer. LAMHA reuses that framework twice: to build the multilingual evaluation set (perturbed script-shift and code-mixing test cases, not just clean English questions), and

to justify indexing each chunk with a normalized Urdu-script form, a Roman transliteration, and an English gloss so retrieval recall doesn't collapse when a query switches script mid-sentence. Very few candidates can explain, from their own research, *why* their multilingual retrieval design works.

03 Architecture

Query in, answer out, with three decision points along the way: does this need retrieval at all, has this been answered before, and is what came back actually good enough to answer from.



Stage	What it does	Why it earns its place
Query Router	Language ID + intent classification via a single Gemini call (chit-chat vs. factual vs. procedural vs. comparison).	Skips the entire pipeline for greetings and small talk — real cost and latency savings, not a cosmetic feature.
Semantic Cache	Embeds the query, checks similarity against previously answered (and still valid) questions.	Freelancer tax questions repeat constantly; this is where most of the latency win comes from.

Stage	What it does	Why it earns its place
Hybrid Retrieval	Qdrant dense + sparse (BM25/SPLADE) search fused with Reciprocal Rank Fusion, filtered by effective-date metadata.	Dense search alone misses exact section-number and numeric-threshold lookups that sparse search nails.
Rerank	Cross-encoder (or Gemini-based) rerank of the fused top-k.	Cuts noise before it ever reaches the grading step, which keeps the correction loop cheap.
Relevance Grading (CRAG)	Gemini grades each surviving chunk as relevant / ambiguous / irrelevant.	This is the difference between “RAG that retrieves” and “RAG that knows if it retrieved anything useful.”
Query Rewrite / Fallback	On low-confidence grading: rewrite the query and re-retrieve, or fall back to a scoped web search for very recent circulars.	Prevents the system from confidently answering off of garbage context — the single biggest trust problem in production RAG.
Generation	Gemini answers with mandatory section-level citations and an explicit amendment/date check.	Legal and tax answers without a citation and a date are close to useless.

04 Tech stack

Layer	Choice	Why this and not the obvious alternative
LLM	Gemini API (flash for routing/grading, pro for final generation)	Cheap, fast model for the high-frequency router/grading calls; stronger model reserved for the answer that actually matters.
Vector DB	Qdrant (named vectors for dense + sparse, payload filtering on effective dates)	Native hybrid search and payload filtering in one system, instead of stitching a keyword index on the side.
Embeddings	Gemini text-embedding model, multilingual	One embedding call covers Urdu, Roman Urdu, and English without a separate model per script.
Cache	Redis (or a dedicated small Qdrant collection) for embedding-similarity cache	Sub-second repeat-question latency without standing up a separate system just for caching.
Backend	FastAPI, async, streamed responses (SSE)	Already the stack from the FYP and the internship — no new framework risk on the one thing that has to actually work.

Layer	Choice	Why this and not the obvious alternative
Frontend	Minimal Next.js (or Streamlit for the first cut) chat UI with citation + “effective as of” badges	Enough to demo convincingly on video; not the place to spend the scarce time budget.
Orchestration	LangGraph for the router / retrieve / grade / rewrite loop	Explicit graph state makes the corrective loop inspectable — exactly what a reviewer will ask about in an interview.
Eval	RAGAS (faithfulness, answer relevancy, context precision/recall) + a custom script-shift/code-mix eval set	RAGAS covers the standard metrics; the custom set is what proves the multilingual claim with numbers instead of a slide.
Deployment	Docker Compose locally; Qdrant Cloud free tier; backend on Render/Fly.io; frontend on Vercel	Fully free-tier, fully live, fully linkable from a resume — no “clone the repo to see it work.”
CI	GitHub Actions running the eval harness on every PR	Regression-tests answer quality, not just that the code compiles — a genuinely senior habit almost no student project has.

05 Data sources

- Income Tax Ordinance, 2001 (as amended) — primary statute, public PDF from FBR.
- Sales Tax Act, 1990 and relevant Sales Tax Rules — for freelancers registered for sales tax on services.
- FBR SROs and circulars from the last 3–5 years — this is where most amendments and exemptions actually live, and where date-tracking matters most.
- State Bank of Pakistan Foreign Exchange Manual, chapters covering IT/ITeS export proceeds and freelancer remittances — the second half of the freelancer compliance question that pure tax law doesn’t cover.
- PSEB freelancer registration and tax-credit guidance documents.
- A hand-built evaluation set of ~120–150 realistic code-switched questions (Roman Urdu / Urdu / English), written using the script-shift and code-mixing-intensity framing from the multilingual robustness paper rather than invented ad hoc.

All of the above are public government/regulatory documents; no scraping of personal or gated data is required, which keeps the whole pipeline legally uncomplicated to publish and demo.

06 Evaluation plan

The eval harness is not an afterthought here — it’s the thing that turns “I built a RAG system” into “I built a RAG system and here are the numbers that prove it’s better than the naive version.” Build the naive baseline first (Week 2 below), then measure every subsequent change against it.

Metric	Measures	Target framing
Faithfulness (RAGAS)	Is the answer actually supported by retrieved context?	Naive baseline vs. corrective pipeline, same question set
Context precision / recall (RAGAS)	Is retrieval pulling the right chunks, and only the right chunks?	Before/after hybrid search + rerank
Retrieval recall@5 / @10	Raw retrieval quality pre-generation	Dense-only vs. hybrid dense+sparse
Script-shift / code-mix accuracy	Does answer quality hold up when the same question is asked in a different script or mixing ratio?	Custom eval set, styled on the paper's SS/ST protocol
Latency P50 / P95	End-to-end response time	Cache-cold vs. cache-warm; naive vs. cached+routed pipeline
Cache hit rate	Share of queries served without a full pipeline run	Measured over the realistic FAQ-style eval traffic
Cost per query	Gemini token spend per answered question	Router+cache savings vs. always-generate baseline

07 Deployment

- Local dev: Docker Compose spinning up FastAPI backend, Qdrant, and Redis together — one command to run the whole stack.
- Qdrant Cloud free cluster for the live deployment; collections use binary or scalar quantization to keep the free tier's memory limit comfortable and retrieval fast.
- Backend deployed on Render or Fly.io (free tier), serving streamed responses over SSE.
- Frontend deployed on Vercel — a small chat UI showing the answer, its citations, and an "effective as of [date]" badge so the time-awareness claim is visible, not just internal.
- GitHub Actions runs the RAGAS + custom eval set on every pull request and fails the build if faithfulness or recall regress past a threshold — this single habit is worth more in an interview than another feature.

08 Repository structure

```
lamha/
■■ ingestion/          # PDF parsing, clause-aware chunking, metadata tagging
■■ indexing/           # embedding + Qdrant collection management
■■ pipeline/
■  ■■ router.py        # language ID + intent classification
■  ■■ retrieve.py      # hybrid search, RRF fusion, date filtering
■  ■■ rerank.py
■  ■■ grade.py         # corrective RAG relevance grading
```

```
■  ■■ rewrite.py       # query rewrite + web-search fallback
■  ■■ generate.py      # citation-grounded answer generation
■■ cache/              # semantic cache read/write
■■ eval/
■  ■■ ragas_suite.py
■  ■■ codeswitch_eval_set.jsonl
■■ api/                # FastAPI app + SSE streaming endpoints
■■ frontend/           # chat UI
■■ docker-compose.yml
■■ .github/workflows/eval.yml
■■ README.md           # architecture diagram, eval table, demo link
```

09 Six-week build roadmap

Week	Focus	Done when…
1	Ingestion + chunking + indexing	All source documents parsed into clause-aware chunks with effective-date metadata and indexed in Qdrant.
2	Naive baseline + eval harness v0	A plain retrieve-then-generate pipeline exists end to end, and the RAGAS eval harness produces a first baseline score — this is your control group.
3	Hybrid retrieval + rerank + router	Dense+sparse fusion and reranking measurably beat the dense-only baseline on recall; router correctly skips retrieval on chit-chat.
4	Corrective loop + semantic cache	Low-confidence retrieval reliably triggers rewrite/fallback; repeat questions are served from cache in under a few hundred milliseconds.
5	Multilingual eval set + temporal logic + deployment	Script-shift/code-mix eval set built and run; amendment/date-check logic verified against known superseded clauses; app live on Render/Fly.io + Vercel.
6	Polish, CI, README, demo	GitHub Actions running eval on PRs; README with architecture diagram and eval numbers; 60–90 second demo video recorded.

10 What goes on the resume / LinkedIn / GitHub

Resume bullet templates (fill in the bracketed numbers once the eval harness has run):

- *Designed and shipped LAMHA, a multilingual retrieval-augmented system answering Urdu / Roman Urdu / English tax-compliance questions against continuously amended regulatory law, with citation-grounded, date-aware answers.*

• *Built a corrective-RAG loop (Gemini-based relevance grading + query rewrite + scoped web fallback) that improved answer faithfulness by [X]% over a naive retrieve-then-generate baseline, measured with RAGAS.*

• *Implemented hybrid dense+sparse retrieval on Qdrant with effective-date filtering and a semantic cache layer, cutting median response latency by [X]% and serving [X]% of repeat queries in under [X] ms.*

LinkedIn / GitHub README pitch (one paragraph): “Most RAG demos answer clean English questions about a static PDF. LAMHA answers real, code-switched questions about tax law that changes underneath it — with a corrective loop that catches bad retrieval instead of confidently hallucinating, and a cache layer that makes the whole thing fast enough to actually use.”

For the GitHub README specifically: lead with the architecture diagram, put the eval numbers table right under it, and link a 60–90 second demo video before any setup instructions. That ordering is what gets a recruiter past the first ten seconds.

11 Risks, and what to cut if time runs short

Risk	Mitigation / fallback
Law text is long and clause numbering is inconsistent across documents, making structural chunking harder than expected.	Fall back to recursive character/token chunking with section-number regex as a best-effort metadata tag, not a hard requirement.
Web-search fallback for very recent circulars adds an external dependency and possible cost.	Cap fallback calls per session; make it clearly optional and label its output as unverified against the primary corpus.
Building a genuinely realistic code-switched eval set takes real time.	Start with 40–50 hand-written questions styled on the paper’s case-study framework; expand later rather than blocking on a large set.
Six weeks is tight alongside coursework.	If something has to be cut, cut the frontend polish and the web-search fallback first — keep the router, hybrid retrieval, corrective loop, cache, and eval harness, since those are what make the project defensible in an interview.

12 Stretch goals, once the core works

- Multi-tenant domain swap: point the same pipeline at labor law or consumer-rights documents to show the architecture generalizes, not just the one dataset.
- Voice input: reuse audio-pipeline experience from the Security.PK FYP to accept a spoken Roman Urdu/Urdu question and transcribe before routing.
- Fine-tune a small reranker on the custom eval set once it’s large enough, and report the lift over the off-the-shelf cross-encoder.
- A small public dashboard showing live cache hit rate and latency percentiles — turns an internal metric into something a visitor to the demo can actually see working.